

2024 Fall (New)

1.a) Describe the architecture of SIC and SIC/XE machine
= SIC :

→ It is a hypothetical computer designed for teaching system software concepts such as assemblers, loaders and linkers. It's a 24 bit machine with simple set & architecture.

i) Memory : 2^{15} bytes = 32kB

ii) Registers : It has 5 registers, each of 24 bits :

Accumulator (A) → performs arithmetic & logical operations

Index register (X) → used for indexed addressing

Linkage register (L) → stores return address during subroutine calls

Program Counter (PC) → holds address for next instruction.

Status word (SW) → contains condⁿ codes & processor status.

iii) Instruction Formats.

8-bits	16 bit	15 bits
Opcode	X	Add ^r

iv) Addressing modes: Direct, indirect

SIC-XE :

→ It is enhanced version of SIC. It is upwards compatible i.e. programs written for SIC also executes in SIC-XE without modification. It adds more memory, registers, addressing modes & instruction formats.

i) Memory : 2^{20} bytes = 1MB

ii) Registers :

Accumulator A 0

Index register X L

Linkage register	L	2
Base register	B	3
General Purpose register	S	4
" " "	T	#5
Floating point register	F	6
Program counter	PC	8
Status word	SW	9

iii) Instruction format :

• Format 1 :

opcode

8-bits

• Format 2 :

opcode	r1	r2
--------	----	----

8-bits 4-bits 4-bits

• Format 3 :

opcode	n	i	x	b	p	e	displacement
--------	---	---	---	---	---	---	--------------

~~6~~ 6 1 1 1 1 1 1 12

• Format 4 :

opcode	n	i	x	b	p	e	addr
--------	---	---	---	---	---	---	------

6 1 1 1 1 1 1 20

iv) Addressing modes :

n i x b p e

0 → format 3

1 → format 4

1 → PC relative

1 → base relative

1 → indexed

1 1 → neither indirect nor

0 1 → immediate immediate

1 0 → indirect

1.6) Write SIC program to find sum of two variables & store in a variable.

LABEL	OPCODE	OPERAND	OBJ CODE
SUM	START	0000	
	LDA	NUM 1	
	ADD	NUM 2	
	STA	RESULT	
ALPHA	WORD	6	
BETA	WORD	7	
RESULT	RESW	1	
	END	SUM	

2.a)

line	LOCN	LABEL	OPCODE	OPERAND	OBJ CODE
10		SUM	START	0	—
20	0000	FIRST	CLEAR (B)	X	B410
30	0002		LDA (00)	#0	010000
40	0005		TLDB (00)	#TOTAL	6910178B
50	0009		BASE	TOTAL	—
60	0009	LOOP	ADD (11)	TABLE, X	1BA00F
70	000C		TX (20)	COUNT	2F2009
80	000F		JLT (07)	LOOP	3B2FF7
90	000F2		STA (00)	TOTAL	0F4000
110	0015		RSUB (40)		4F0000
120	0018	COUNT	RESW	1	—
130	001B	TABLE	RESW	2000	—
150	178B	TOTAL	RESW	1	—
160	178E		END	FIRST	—

20) CLEAR X : 13410

30) LDA #0

00

i) Data structures:

OPTAB

SYMTAB

FIRST	0000
Loop	0009
COUNT	0018
TABLE	001B
TOTAL	178B

CLEAR	B4	STA	0C
LDA	00	ASUB	4C
LDB	68		
ADD	18		
TIx	2C		
JLT	38		

20) CLEAR x → B410

30) LDA #0

00 0000

0000 0001 0000 0000 0000 0000 0000 = 010000

40) +LDB #TOTAL

68 178B

0110 1001 0001 0178B = 6910178B

60) ADD TABLEx

18 001B

0001 1000 1010 00F = 100000F

Displacement = 001B - 000C = 000F

70) TIx COUNT

2C 0018

0010 1111 0010 009 = 2F2009

Disp = 0018 - 000F = 0009

80) JLT LOOP

38 0009

0011 1011 0010 00FF7 = 3B2FF7

Disp = 0009 - 0012 = FF7

90) STA TOTAL

0C 178B

0000 1111 0100 000 = 0F2000 of 4000

Disp = 178B - 0A5 = 1776 X

So Disp = 178B - BASE = 178B - 178B = 0000

Object program:

H ~~000000~~ SUM --- ^ 000000 ^ 00178E

T ^ 000000 ^ 1P ^ B410 ^ 010000 ^ 6910178B ^ 1B1A00F ^

2F2009 ^ 8B2FF7 ^ 0F4000 ^ 4F0000

E ^ 000000

2.5) Explain the control section and program linking.

= A ~~part~~ control section is an independent block of program that can be assembled, relocated and loaded separately.

Each control section has ~~its~~ :

→ its own name, location counter

→ Can be loaded into different memory locations

→ can refer to symbols in other control sections using external references.

→ Advantages:

• Easier debugging

• Faster compilation

• Easier maintenance

• Code reusability

external definition (EXTDEF): A symbol that is defined inside one control section and can be used by other control sections.

Eg: EXTDEF BUFFER

External reference (EXTREF): A symbol used inside one control section but defined in another.

Eg: EXTREF BUFFER.

Program Linking:

→ It is the process of combining two or more object programs into a single executable program. It is performed by the linker.

Steps:

i) Assemble each module separately

ii) Generate object programs

iii) Link all object programs

iv) Resolve external dependencies.

v) Load into memory.

Eg :

Module A : MAIN
 + JCRB MOD

Module B : ADD

Assembler can't find ADD, so the linker later connects MAIN with ADD.

3.a)

Line	LOC	Symbol	OpCode	Exp	Obj code
10		TEST	START	3000	—
20	3000	FIRST	LOA	NUM	00300F
30	2003		MUL	TEN	203030
40	3006		DIV	FIVE	243033
50	3009		ADD	SIX	183036
60	300C		STA	ANS	0C3020
70	300F	NUM	RESW	10	—
80	3020	ANS	RESW	1	—
90	3030	TEN	WORD	10	00000A
100	3033	FIVE	WORD	5	000005
110	3036	SIX	WORD	6	000006
120	3039		END		—

Obj code file

TH TEST -- ^ 003000 ^ 000039

T ^ 003000 ^ 18 ^ 00300F ^ 203030 ^ 243033 ^ 183036 ^
0C3020 ^ 00000A ^ 000005 ^ 000006

Loading in memory:

	00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F	xx xx xx xx xx xx xx xx xx xx xx xx xx xx xx xx
3000		
3000	00 30 0F 20 30 30 24 30 33 18 30 36 0C 30 2D xx	
3030	00 00 0A 00 00 05 00 00 06 xx xx xx xx xx xx xx xx xx xx	

4 a) What problem arises when a label is declared inside a macro body? How can this problem be solved? Illustrate with an example.

= When a label is declared inside a macro body, a problem occurs because the macro may be expanded multiple times. Each expansion generates a label name, causing duplicate label definitions.

Problem:

MAC1	MACRO	&ch	When Invoked multiple times
Repeat	JEQ	Repeat	Repeat JEQ Repeat
	LDCH	#&ch	LDCH #&ch
	MEND		Repeat JEQ Repeat
			LDCH #&ch

The assembler reports an error because Repeat is defined twice. It can't determine which Repeat a branch instruction should refer to.

Solⁿ:

The assembler generates a unique label for each macro expansion. This is done using unique label generation.

Eg:

```
MAC1 MACRO
$LOOP LDA VALUE
      ADD ONE
      JLT $LOOP
```

1st expansion:

```
$AALoop LDA VALUE
      ADD ONE
      JLT $AALoop
```

2nd expansion:

```
$ABLoop LDA VALUE
      ADD ONE
      JLT $ABLoop
```

4.6) Differentiate betⁿ macro & sub routine. Consider the macro defn given below and show the macro expansion of the call statement "Print 77 88" and also show call data structure.

PRINT	MACRO	&NUM1, &NUM2
\$LOOP	TD	&NUM2
	JEQ	\$LOOP
	LDCH	#02 NUM1
	WD	&NUM2
	MEND	

MACRO

SUB Routine

- | | |
|---|---|
| i) A sequence of instructions that is a separate block of code that is expanded inline by macroprocessor called during program execution before assembly. | |
| ii) Expansion happens at compilation | Execution happens at run time |
| iii) Increases the size of obj program because code is duplicated at every invocation. | Reduces program size because some code is reused throughout calls. |
| iv) Faster execution since there is no call or return overhead. | Slightly slower due to overhead of calling and returning from sub routine |
| v) Parameters are substituted during expansion | Parameters are passed at run time through registers or memory. |

PRINT 77 FA

DEFTAB:

```

PRINT MACRO INUM1, &NUM2
$AALoop TD FA?2
        JEQ $AALoop
        LDCH #1
        WD 1?2
MEND

```

NAMETAB:

PRINT |*|*

ARGTAB:

1	77
2	FA

```

$AALoop TD FA
        JEQ $AALoop
        LDCH #77
        WD 2 DFA
MEND

```

i. a) Write a basic sample program in HLL and show how it is compiled through various compiler phases.

= Suppose sample program: `int result = a + b * c;`

a) Lexical Analysis:

→ It reads the HLL program one character at a time and groups them into meaningful units called tokens.

LEXEMES TOKEN

int KEYWORD

result IDENTIFIER

= ASSIGNMENT

a IDENTIFIER

+ ADDITION

b IDENTIFIER

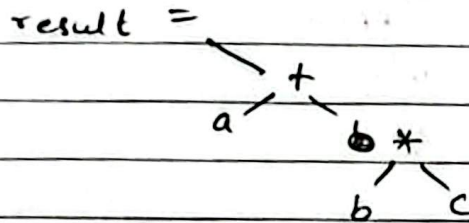
* MULTIPLICATION

c IDENTIFIER

; SEMI COLON

b) Syntax analysis:

- It takes token as input and produces parse tree. It's main function is to check whether statement is prescribed by language grammar.



c) Semantic analysis:

- It takes syntax parse tree as input and checks ~~meaning~~ ^{meaning} of program after syntax is verified.

Functions:

- checks declaration of variables
- Type checking
- Array index validation
- Scope checking
- Function argument checking

d) Intermediate code:

- It generates intermediate representation that is independent of target machine.

$$t_1 = b * c$$

$$t_2 = a + t_1$$

$$\text{result} = t_2$$

- It generally uses Three address code strategy to generate intermediate representation.

Advantages:

- It makes the optimization process faster, simplifies code generation and allows machine independent compilation.

e) Code optimization:

- The main purpose of this is to improve the intermediate code so that the final program executes faster using fewer resources.

Common techniques: constant folding, constant propagation, loop optimization, dead code optimization, common expression elimination etc.

f) Code generation:

→ It generates machine dependent code based on optimised code.

```
LDA    a
ADD    b
MUL    c
STA    result
```

5.b) Explain Booch's macro & micro processes.

i) Booch's macro process:

→ It is the high level process used to manage the entire software development project. It describes how the software evolves from requirements to deployment.

Phases of Booch's macro process:

a) Conceptualization:

→ This is the initial planning phase where the project's purpose is defined.

Activities:

- identify the problem
- define project goals
- determine scope
- identify major requirements
- evaluate feasibility

Output: project vision, initial requirements.

b) Analysis:

The system requirements are studied to understand what the system should do.

Activities:

- Gather requirements
- Identify actors and use cases
- Identify objects & classes
- Define relationship between objects.

Output: analysis model, use case diagrams, ^{initial} class diagrams

c) Design:

→ The solution is designed using object oriented principles.

Activities:

- Refine class
- Design class hierarchy
- Define inheritance
- Define interfaces
- Design object interactions
- Design database & architecture

Output: detailed class diagrams, sequence diagram, collaboration diagram

d) Implementation:

→ The design is converted into actual software.

Activities:

- Coding, unit testing, integration, debugging.

e) Maintenance:

→ After deployment, the software is maintained.

Activities:

- Fix bugs, Add new features, improved performance, update documentation

ii) Booch's micro process:

→ It focuses on developing and refining individual classes & objects. It is repeated many times during the macro process.

Steps of micro process:

a) Identify the classes & objects:

→ Determine which objects exist in the problem domain.

Eg in LMS: Book, Student, librarian, library.

b) Identify semantics:

- Define what each class represents.

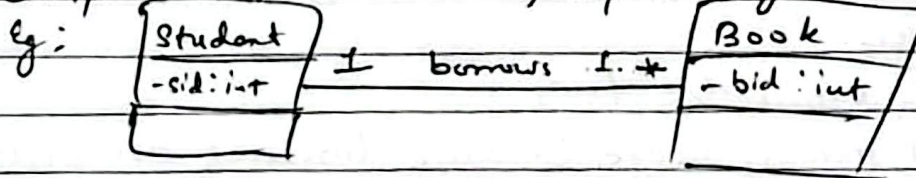
Eg: Book : - Title
 - Author

Responsibilities: Borrow, return, reserve.

c) Identify relationships:

- Determine how classes are connected.

Relationships include: association, aggregation, composition, inheritance, dependency.



d) Specify interfaces:

- Define how objects communicate.

Eg: + borrowBook(), + returnBook() etc.

e) Implement the class:

- The code is written using any OOP language.

```
class Book {
```

```
    public private:
```

```
        String title;
```

```
    public:
```

```
        void borrow() { }
```

```
        void returnBook() { }
```

```
}
```

f) Test the class:

- Ensure the class behaves correctly.

Testing includes: constructors, methods, object interactions, exception handling.

Describe various loader design options.

Dynamic Linker Linking:

In dynamic linking, linking of library routines is postponed until the program is executing.

Working:

- The executable contains references to shared libraries.
- When a library routine is first called, the OS loads and links it.
- The linked routine can then be reused for subsequent calls.

Advantages:

- Smaller executable files
- Reduced memory usage because libraries are shared
- Library updates do not require recompiling applications.

Disadvantages:

- Slight runtime overhead
- Execution may fail if a required library is unavailable.

Linkage editor:

A linkage editor performs linking before the program is loaded and creates an executable file.

Working:

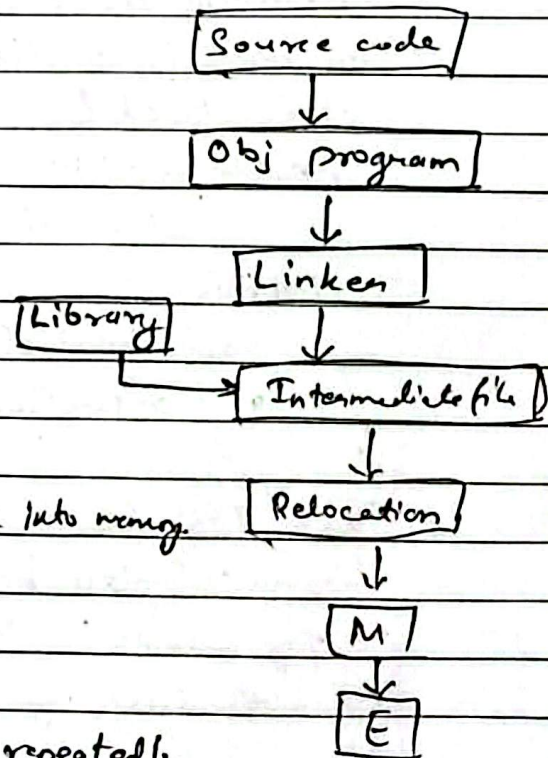
- Combines object modules once.
- Resolves external references
- Produces a linked executable
- The loader only loads the executable into memory.

Advantages:

- Faster program loading
- Linking is only done only once
- Efficient for programs executed repeatedly.

Disadvantages:

- If any module changes, the executable must be rebuilt.

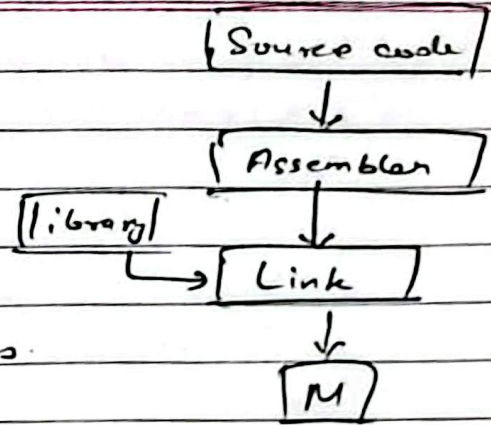


Linking loader:

A linking loader performs both linking and relocation at load time.

Working:

- Loads object modules into memory.
- Resolves external symbol references.
- Performs relocation.
- Starts program execution.



Advantages:

- No separate linker required
- Supports modular programming
- Programs can be loaded at different memory locations

Disadvantages:

- Longer loading time.
- Linking is repeated every time the program is executed.

Assembler design options:

An assembler translates an assembly language program into machine language. Depending on how it processes the source program, different assembler design options are used.

Single pass assembler:

It scans the source program only once to generate the object code.

Working:

- Reads each statement only once
- Builds the symbol table as the labels are encountered.
- Generates object code immediately.
- Uses techniques such as forward reference lists to handle symbols that are defined later.

Advantages:

- Faster assembly
- Less processing time
- Suitable for small programs

Disadvantages:

- Difficult to handle forward references.
- More complex implementation.

ii) Two pass assembler:

→ A two pass assembler processes the source program twice
Pass 1:

- Assign addresses using the location counter
- Creates the symbol table
- Processes assembler directives.

Pass 2:

- Uses the symbol table to resolve addresses.
- Generates obj code.
- Produces the obj program.

Advantages:

- Easily handles forward references.
- Simpler and more reliable than a single pass
- Most widely used.

Disadvantages:

- Takes more time because the source program is read twice

Q

iii) Multi pass assembler.

→ It processes the source code more than two times until all addresses and symbol references are correctly resolved.

Working:

- Performs multiple passes over the source program
- Each pass resolves additional information that could not be determined in earlier passes.
- Stops when no unresolved references remain.

Advantages:

- Handles very complex assembly programs.
- Produces accurate obj code for complicated programs
- Suitable for advanced features such as nested macros, complex expressions etc.

Disadvantages:

- Slowest among all assembler designs
- Requires more processing time and memory

Concatenation of macro parameters is a macroprocessor technique in which a macro parameter is joined with other characters or symbols to form a new identifier, label or variable name during macro expansion.

Purpose:

- Generate unique symbol names.
- Create labels or variables dynamically.
- Avoid name conflicts in macro expansions.

Eg:

```
MACRO INCR &NUM  
    LDA    VALUE&NUM  
    ADD    ONE  
    STA    VALUE&NUM  
MEND
```

MACRO call : INCR 1

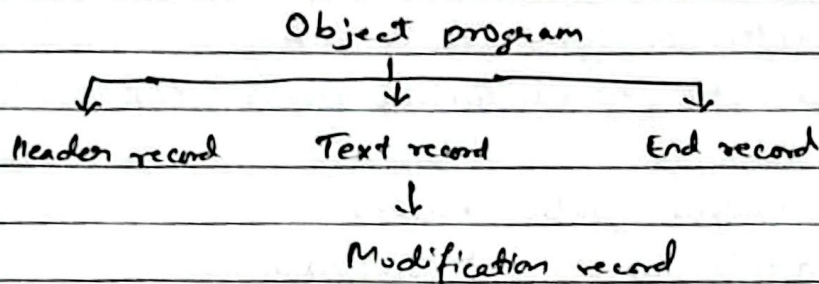
Expanded code:

```
LDA    VALUE1  
ADD     ONE  
STA    VALUE1
```

Advantages:

- Creates meaningful and unique identifiers
- Increase macro flexibility
- Reduces repetitive coding.

6.a) Explain the object diagram for assembler with a diagram
=



→ An object program is the output produced by an assembler. It contains the machine code and additional information required by loader, such as relocation and execution details.

i) Header record (H):

→ Contains general information about the program.

Format:

H n ProgramName n Start add^r n program length

Eg:

H n SUM--- n 001000 n 000051

ii) Text record (T):

→ Contains the actual machine (object) code.

Format:

T n start add^r n length n Object codes

Eg: T n 001000 n 1E n 005062 n 2F1000 n 4F0000 n ...

iii) Modification record (M):

→ Used when relocation is reqd. It tells the loader which add^r need to be modified if the program is loaded at different location.

Format: M n address n length

Eg: M n 001007 n 05

iv) End record (E):

→ Indicates the address of the first instruction to execute.

Format: E n ExecutionAdd^r

Eg: E n 001006